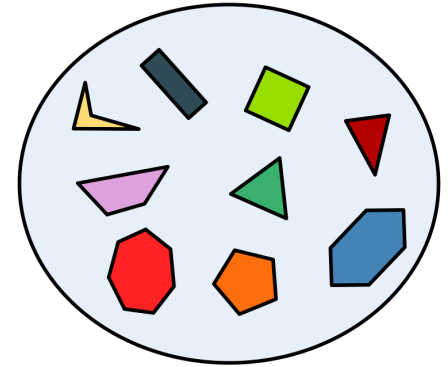


Sets

Eunji Kim

eunjikim@cau.ac.kr



Set

Set is an unordered unindexed collection of unique items.

 To store multiple items in a single variable, we can use a List, Tuple, Set, or Dictionary, each with distinct uses.

Sets are created with curly brackets:

```
thisset = {"apple", "banana", "cherry"}  
print(thisset)
```

Duplicates Not Allowed:

```
thisset = {"apple", "banana", "cherry", "apple"}  
print(thisset) # {"apple", "banana", "cherry"}
```

It is also possible to use the `set()` constructor to make a set:

```
mytuple = ("apple", "banana", "cherry")
thisset = set(mytuple)
print(thisset)
```

To determine how many items a set has, use the `len()` function:

```
thisset = {"apple", "banana", "cherry"}
print(len(thisset)) # 3
```

Access Set Items

Set items are *unordered*. You cannot access items in a set by referring to an index or a key.

But you can loop through the set items using a for loop, or ask if a specified value is present in a set, by using the `in` keyword.

```
thisset = {"apple", "banana", "cherry"}

for x in thisset:    # Loop through the set, and print the values
    print(x)         # The printing order may vary.

print("banana" in thisset) # True
print("kiwi" not in thisset) # True
```

Change Items

Sets are immutable (Unchangable):

Once a set is created, you cannot change its items, but you can add new items.

Add Items

To add one item to a set use the `add()` method.

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.add("orange")  
print(thisset) # {"apple", "orange", "banana", "cherry"}
```

Add Sets

To add items from another set into the current set, use the `update()` method.

```
thisset = {"apple", "banana", "cherry"}  
tropical = {"pineapple", "mango", "papaya"}  
  
thisset.update(tropical)  
print(thisset) # {'apple', 'mango', 'cherry', 'pineapple', 'banana', 'papaya'}
```

Add Any Iterable

The object in the `update()` method does not have to be a set, it can be any iterable object (tuples, lists, dictionaries etc.).

```
thisset = {"apple", "banana", "cherry"}  
mylist = ["kiwi", "orange"]  
  
thisset.update(mylist)  
print(thisset) # {'banana', 'cherry', 'apple', 'orange', 'kiwi'}
```


Remove Item

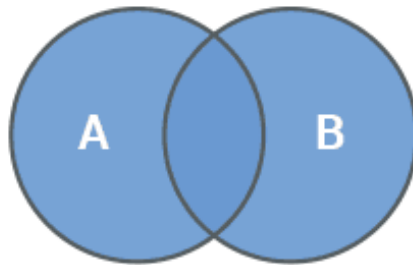
To remove an item in a set, use the `remove()` , or the `discard()` method.

```
thisset = {"apple", "banana", "cherry"}  
  
thisset.remove("banana")  
print(thisset) # {"apple", "cherry"}  
  
thisset.discard("apple")  
print(thisset) # {"cherry"}
```

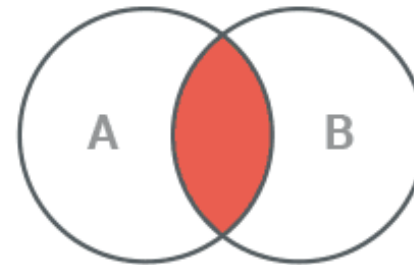
⚠ If the item to remove does not exist, `remove()` or `discard()` will raise an error.

Join Sets

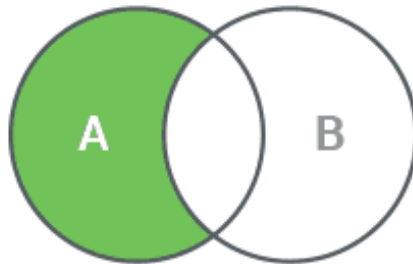
There are several ways to join two or more sets in Python.



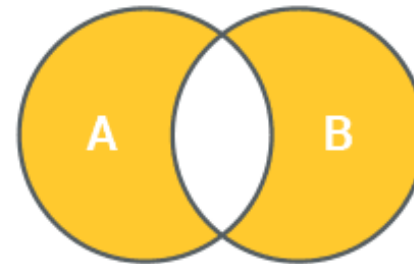
Union



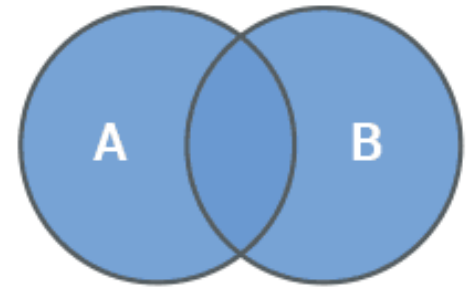
Intersection



Difference



Symmetric Difference



Union

The `union()` method returns a new set with all items from both sets.

```
set1 = {"a", "b", "c"}  
set2 = {1, 2, 3}  
  
set3 = set1.union(set2)  
print(set3) # {'c', 2, 1, 'a', 'b', 3}
```

 You can use the `|` operator instead of the `union()` method, and you will get the same result.

```
set4 = set1 | set2  
print(set4) # {'c', 2, 1, 'a', 'b', 3}
```

 The `union()` method allows you to join a set with other data types, like lists or tuples. The result will be a set.

```
x = {"a", "b", "c"}  
y = (1, 2, 3)  
  
z = x.union(y)  
print(z) # {2, 'b', 1, 'a', 3, 'c'}
```

Join Multiple Sets

All the joining methods and operators can be used to join multiple sets.

When using a method, just add more sets in the parentheses, separated by commas:

```
set1 = {"a", "b", "c"}
set2 = {1, 2, 3}
set3 = {"John", "Elena"}
set4 = {"apple", "bananas", "cherry"}

myset = set1.union(set2, set3, set4)
print(myset) # {'a', John, Elena, 'c', 3, 1, 'b', banana, cherry, apple, 2}
```



When using the | operator, separate the sets with more | operators:

```
myset = set1 | set2 | set3 | set4
print(myset) # {'a', John, Elena, 'c', 3, 1, 'b', banana, cherry, apple, 2}
```

Update

The `update()` method inserts all items from one set into another.

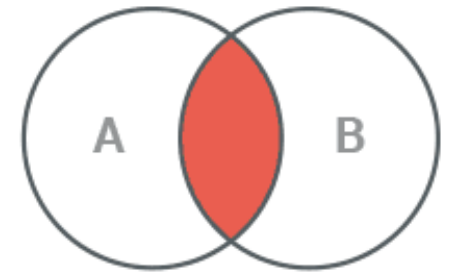
The `update()` changes the original set, and does not return a new set.

The `update()` method inserts the items in `set2` into `set1` :

```
set1 = {"a", "b" , "c"}  
set2 = {1, 2, 3}  
  
set1.update(set2)  
print(set1) # {1, 'c', 'a', 'b', 3, 2}
```



Note: Both `union()` and `update()` will exclude any duplicate items.



Intersection

Keep ONLY the duplicates.

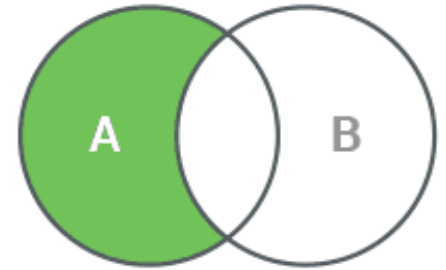
The `intersection()` method will return a new set, that only contains the items that are present in both sets.

Join set1 and set2, but keep only the duplicates:

```
set1 = {"apple", "banana", "cherry"}  
set2 = {"google", "microsoft", "apple"}  
  
set3 = set1.intersection(set2)  
print(set3) # {'apple'}
```

 You can use the `&` operator instead of the `intersection()` method, and you will get the same result.

```
set3 = set1 & set2  
print(set3) # {'apple'}
```



Difference

The `difference()` method will return a new set that will contain only the items from the first set that are not present in the other set.

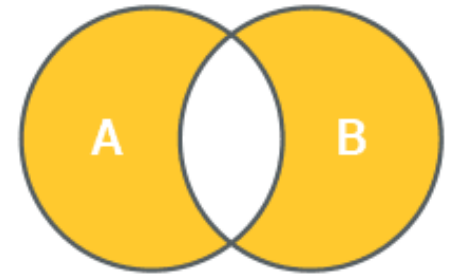
Keep all items from `set1` that are not in `set2` :

```
set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set3 = set1.difference(set2)
print(set3) # {'banana', 'cherry'}
```

 You can use the `-` operator instead of the `difference()` method, and you will get the same result.

```
set3 = set1 - set2
print(set3) # {'banana', 'cherry'}
```

Symmetric Differences

The `symmetric_difference()` method will keep only the elements that are NOT present in both sets.

Keep the items that are not present in both sets:

```
set1 = {"apple", "banana", "cherry"}
set2 = {"google", "microsoft", "apple"}

set3 = set1.symmetric_difference(set2)
print(set3) # {'google', 'banana', 'microsoft', 'cherry'}
```

 You can use the `^` operator instead of the `symmetric_difference()` method, and you will get the same result.

```
set3 = set1 ^ set2
print(set3) # {'google', 'banana', 'microsoft', 'cherry'}
```

Thank You.